

EXCEL ENGINEERING COLLEGE
(Autonomous)
KOMARAPALAYAM – 637303

DEPARTMENT OF COMPUTER SCIENCE AND BUSINESS SYSTEMS



23CB502 –AUTOMATA AND COMPILER DESIGN LABORATORY

USER MANUAL

EXCEL ENGINEERING COLLEGE
KOMARAPALAYAM

VISION AND MISSION STATEMENTS OF INSTITUTE

VISION

To create competitive human resource in the fields of engineering for the benefit of society to meet global challenges.

MISSION

To provide a conducive ambience for better learning and to bring creativity in the students

To develop sustainable environment for innovative learning to serve the needy

To meet global demands for excellence in technical education

To train young minds with values, culture, integrity, innovation and leadership



**EXCEL ENGINEERING COLLEGE
KOMARAPALAYAM**



DEPARTMENT OF CSBS

Vision of Department

To create better quality technical engineers in computer science and engineering with ethically strong values which cater local and global needs of the society.

Mission of Department

- To instill quality in engineering education that demands excellence
- To initiate desires among the students to work in close cooperation and collaboration with industry and professional bodies
- To train the students for developing software and novel software systems
- To create ambience for taking initiatives towards entrepreneurship and lifelong learning



EXCEL ENGINEERING COLLEGE

KOMARAPALAYAM



DEPARTMENT OF CSBS

PROGRAMME EDUCATIONAL OBJECTIVES (PEOs)

- I. To provide fundamental knowledge to formulate, solve, analyze engineering problems and to pursue higher studies
- II. To develop the ability of the students in comprehending, analyzing and synthesizing data in order to design software and to create novel software systems
- III. To inculcate effective communication skills, team skills, professional and ethical attitude in the students for enabling them to relate engineering issues with social issues in a broader context
- IV. To provide students managerial and leadership skills so as to make them successfully employed and to demonstrate a pursuit of lifelong learning in multidisciplinary environment



EXCEL ENGINEERING COLLEGE KOMARAPALAYAM



DEPARTMENT OF CSBS

PROGRAMME OUTCOMES [Pos]

Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.

Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

Design / Development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

Conduct investigations of complex problems: Use research-based knowledge and research methods, including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling of complex engineering activities with an understanding of the limitations.

The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

Environment and Sustainability: Understand the impact of the professional engineering solutions to societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

Individual and team work: Function effectively as an individual and as a member or leader in diverse teams, and in multidisciplinary settings.

Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

Project management and finance: Demonstrate knowledge and understanding of the engineering management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

Lifelong learning: Recognize the need for and have the preparation and ability to engage in independent and lifelong learning in the broadest context of technological change.



EXCEL ENGINEERING COLLEGE
KOMARAPALAYAM
DEPARTMENT OF CSBS



PROGRAMME SPECIFIC PROGRAMME OUTCOMES

1. An ability to learn about recent trends in all domains to solve the real world problems
2. To play a vital team role to enrich their design and development skills

List of Exercises:

S.No	List of Exercises	CO Mapping	RBT
1	Design a DFA over the input set (a,b) accept all the sting starting with symbol a	CO1	Apply
2	Write the program to implement the conversion from NFA to DFA	CO1	Apply
3	Implementation of a DFA Using Transition Table in C for Recognizing given string.	CO2	Apply
4	Write program that simulates a Pushdown Automaton (PDA) for the context-free language.	CO2	Apply
5	Using the LEX tool, Develop a lexical analyzer to recognize a few patterns in C. (Ex identifiers, constants comments, operators etc.). Create a symbol table, while recognizing identifiers	CO3	Apply
6	i.Implement lexical analyzer using LEX Tool ii.Implementation of Calculator using LEX and YACC	CO4	Apply
7	Implement Shift Reduce Parsing	CO5	Apply
8	Implement Predictive parsing algorithm	CO5	Apply
9	Implement Intermediate code generation Postfix, Prefix (Infix to Postfix)	CO6	Apply

EX:1 Design a DFA over the input set (a,b) accept all the sting starting with symbol a

AIM:

To design and implement a Deterministic Finite Automaton (DFA) over the input set {a, b} that accepts all strings starting with the symbol 'a' using the C programming language.

ALGORITHM:

1. Start in the initial state q_0
2. Read the input string symbol by symbol.
3. Apply the transition rules:
 - From q_0 , if the first character is:
 - 'a', move to q_1
 - 'b', move to trap state q_2
 - From q_1 , remain in q_1 for any 'a' or 'b'
 - From q_2 , remain in q_2 for any 'a' or 'b'
4. If the final state is q_1 , accept; otherwise, reject

Program:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {  
    char str[100];  
    int state = 0; // q0  
    printf("Enter a string over {a, b}: ");  
    scanf("%s", str);  
    for (int i = 0; i < strlen(str); i++) { char  
        ch = str[i];  
        switch (state) {  
            case 0: // q0  
                if (ch == 'a')  
                    state = 1; // move to q1 else  
                    state = 2; // move to trap q2 break;  
            case 1: // q1  
                if (ch == 'a' || ch == 'b') state  
                    = 1; // stay in q1  
                break;  
            case 2: // trap state q2 state = 2;  
                // remain in q2 break;  
        }  
    }  
}
```

```
    }  
}  
  
if (state == 1)  
    printf("String is accepted by the DFA.\n"); else  
    printf("String is rejected by the DFA.\n"); return 0;  
}
```

Output

Input: abba

Output: String is accepted by the DFA. Input:

bbaa

Output: String is rejected by the DFA.

RESULT:

The DFA was successfully designed and implemented in C. It correctly accepts all strings that start with 'a' over the alphabet {a, b}, and rejects all other strings.

EX:2 Write the program to implement the conversion from NFA to DFA

AIM:

To implement a C program that converts a given NFA into an equivalent DFA using subset construction method.

ALGORITHM :

1. Read number of NFA states and transitions for inputs 0 and 1.
2. Store NFA transitions in a 3D array.
3. Represent each DFA state as a subset of NFA states.
4. Use a queue-like approach to process new DFA states.
5. For each DFA state and input, find all reachable NFA states.
6. Store new DFA transitions in a table.
7. Print the resulting DFA transition table.

Program:

```
#include <stdio.h>
#include <string.h>
#define MAX 20
int nfa[10][2][10]; // nfa[state][input][list of next states]
int dfa[100][2]; // dfa[state][input] = next DFA state
int dfa_states[100][10]; // stores NFA states in each DFA state
int dfa_state_count = 0;
int is_same_set(int *a, int *b) {
    for (int i = 0; i < 10; i++)
        if (a[i] != b[i]) return 0;
    return 1;
}
int is_new_state(int *set) {
    for (int i = 0; i < dfa_state_count; i++) {
        if (is_same_set(dfa_states[i], set)) return i;
    }
    return -1;
}
void print_set(int *set) {
    printf("{");
    for (int i = 0; i < 10; i++) {
        if (set[i] == 1)
            printf("q%d ", i);
    }
    printf("}");
}
int main() {
    int nfa_state_count, trans, state, input, dest, i, j, k;
    printf("Enter number of NFA states: ");
    scanf("%d", &nfa_state_count);
    // Initialize NFA transitions
    for (i = 0; i < nfa_state_count; i++) {
        for (j = 0; j < 2; j++) {
            printf("Enter number of transitions from q%d on input %d: ", i, j);
            scanf("%d", &trans);
            printf("Enter destination states: ");
            for (k = 0; k < trans; k++) {
                scanf("%d", &dest);
                nfa[i][j][k] = dest;
            }
        }
    }
}
```

```

        nfa[i][j][trans] = -1; // end of list
    }
}
// Start subset construction
int queue[100], front = 0, rear = 0;
int visited[100] = {0};
// Start with state {0}
int initial[10] = {0};
initial[0] = 1; // q0
memcpy(dfa_states[0], initial, sizeof(initial));
queue[rear++] = 0;
dfa_state_count = 1;
while (front < rear) {
    int current = queue[front++];
    for (input = 0; input < 2; input++) {
        int new_state[10] = {0};
        for (i = 0; i < nfa_state_count; i++) {
            if (dfa_states[current][i] == 1) {
                for (j = 0; nfa[i][input][j] != -1; j++) {
                    new_state[nfa[i][input][j]] = 1;
                }
            }
        }

        int index = is_new_state(new_state);
        if (index == -1) {
            memcpy(dfa_states[dfa_state_count], new_state, sizeof(new_state));
            dfa[current][input] = dfa_state_count;
            queue[rear++] = dfa_state_count;
            dfa_state_count++;
        } else {
            dfa[current][input] = index;
        }
    }
}
// Print DFA transition table
printf("\nDFA Transition Table:\n");
printf("State\t0\t1\n");
for (i = 0; i < dfa_state_count; i++) {
    print_set(dfa_states[i]);
    printf("\t%d\t%d\n", dfa[i][0], dfa[i][1]);
}
return 0;
}

```

Output:

```

Enter number of NFA states: 3
Enter number of transitions from q0 on input 0: 1
Enter destination states: 0
Enter number of transitions from q0 on input 1: 2
Enter destination states: 0 1
Enter number of transitions from q1 on input 0: 1
Enter destination states: 2
Enter number of transitions from q1 on input 1: 1
Enter destination states: 1
Enter number of transitions from q2 on input 0: 1
Enter destination states: 2
Enter number of transitions from q2 on input 1: 1
Enter destination states: 2
DFA Transition Table:
State    0    1
{q0 }    0    1

```

{q0 q1 }	2	3
{q0 q2 }	2	1
...		

Result:

The program successfully performs conversion of a given NFA into an equivalent DFA using subset construction and prints the DFA transition table.

EX 3 Implementation of a DFA Using Transition Table in C for Recognizing given string

AIM:

To implement a DFA using a transition table in C that accepts strings starting with the symbol 'a', over the input set {a, b}.

ALGORITHM:

1. Define states and transitions in a transition table.
2. Take input string from the user.
3. Set initial state as q0.
4. For each character in the string:
 - If the character is 'a', use index 0; if 'b', use index 1.
 - Use the transition table to move to the next state.
5. After processing the string:
 - If the current state is the accepting state (q1), **accept** the string.
 - Otherwise, **reject** the string.

Program:

```
#include <stdio.h>
#include <string.h>
int main() {
    // Transition table: rows = states, cols = input symbols (0 for 'a', 1 for 'b')
    int transition[3][2] = {
        {1, 2}, // q0: a->q1, b->q2
        {1, 1}, // q1: a->q1, b->q1
        {2, 2} // q2: a->q2, b->q2 (trap)
    };
    char input[100];
    int state = 0; // start from q0
    printf("Enter a string over {a, b}: ");
    scanf("%s", input);
    for (int i = 0; i < strlen(input); i++) {
        int symbol;
        if (input[i] == 'a')
            symbol = 0;
        else if (input[i] == 'b')
            symbol = 1;
        else {
            printf("Invalid input character '%c'.\n", input[i]);
```

```
        return 1;
    }
    state = transition[state][symbol]; // state transition
}
if (state == 1)
    printf("String is ACCEPTED by the DFA.\n");
else
    printf("String is REJECTED by the DFA.\n");
return 0;
}
```

Output Input:

abba

Output: String is ACCEPTED by the DFA.

Input: baa

Output: String is REJECTED by the DFA.

RESULT:

The DFA was successfully implemented using a transition table in C. It correctly accepts all strings that start with 'a' and rejects others.

EX 4 Write program that simulates a Pushdown Automaton (PDA) for the context-free language

AIM:

To write a C program that simulates a Pushdown Automaton (PDA) to accept the context-free language $L = \{ a^n b^n \mid n \geq 1 \}$ using a stack.

ALGORITHM:

1. Initialize an empty stack.
2. Read the input string.
3. For each character:
 - If a, push onto the stack.
 - If b, pop from the stack.
 - If trying to pop when the stack is empty $\rightarrow$ reject.
4. After reading the entire string:
 - If the stack is empty $\rightarrow$ accept.
 - Otherwise $\rightarrow$ reject.

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#define MAX 100
char stack[MAX];
int top = -1;

void push(char symbol) {
    stack[++top] = symbol;
}

char pop() {
    if (top == -1)
        return 'X'; // special symbol for empty
    return stack[top--];
}

int main() {
    char
    input[MAX]; int
    i, valid = 1;

    int stage = 0; // 0: reading a's, 1: reading b's
    printf("Enter a string (L = { a^n b^n }): ");
    scanf("%s", input);

    for (i = 0; i < strlen(input); i++) {
```



```

if (input[i] == 'a' && stage == 0) {
    push('a');
} else if (input[i] == 'b') {
    stage = 1;
    if (pop() != 'a') {
        valid = 0;
        break;
    }
} else {
    valid = 0;
    break;
}
}

// Final condition: stack must be empty and valid transitions
if (valid && top == -1 && stage == 1)
    printf("String is ACCEPTED by the PDA.\n");
else
    printf("String is REJECTED by the PDA.\n");
return 0;
}

```

Output:

Input: aabb

Output: String is ACCEPTED by the PDA.

Input: aaabbb

Output: String is ACCEPTED by the PDA.

RESULT:

The C program simulates a PDA and successfully accepts strings in the language $L = \{ a^n b^n \mid n \geq 1 \}$ and rejects invalid strings. The use of a stack mimics the behavior of a PDA.

EX 5 Using the LEX tool, Develop a lexical analyzer to recognize a few patterns in C. (Ex identifiers, constants comments, operators etc.). Create a symbol table, while recognizing identifiers

AIM:

To write a LEX program to recognize C language patterns like identifiers, constants, operators, and comments, and build a symbol table to store recognized identifiers.

ALGORITHM:

1. Start
2. Define patterns (regular expressions) in LEX for:
 - Identifiers ([a-zA-Z_][a-zA-Z0-9_]*)
 - Constants ([0-9]+(\.[0-9]+)?)
 - Operators (\+|\-|*|\/|\=|\=|<|>|<=|>=)
 - Comments (\/* for single-line and /*([^*]|\/)**/ for multi-line)
3. In the LEX definitions section, declare a symbol table (array or hash map) for identifiers.
4. On recognizing an identifier, check if it already exists in the symbol table:
 - If not, insert it into the table with attributes (like token type, position).
 - If yes, ignore (already present).
5. For each recognized pattern, print the corresponding token type.
6. Stop of Program.

LEX Program:

```
#include <stdio.h>

#include <string.h>

#include <stdlib.h>

int line = 1;

char *keywords[] = {"int", "float", "char", "return", "if", "else", "while", "for", "void"};

int isKeyword(char *str);

void addToSymbolTable(char *identifier);

%}
%option noyywrap
IDENT [a-zA-Z_][a-zA-Z0-9_]*

NUMBER [0-9]+

FLOAT [0-9]*\.[0-9]+
OP [+|-|*|/=|<|>|=]?

COMMENT ("/*.*"(/[^\*]|\/)*\*/)
%%
{COMMENT} { printf("Comment: %s\n", yytext); }
{"{" { printf("Left Paren: %s\n", yytext); }
"}" { printf("Right Paren: %s\n", yytext); }
{"{" { printf("Left Brace: %s\n", yytext); }
"}" { printf("Right Brace: %s\n", yytext); }
{OP} { printf("Operator: %s\n", yytext); }
{FLOAT} { printf("Float Constant: %s\n", yytext); }
{NUMBER} { printf("Integer Constant: %s\n", yytext); }
{IDENT} {
```

```

if (isKeyword(yytext))
printf("Keyword: %s\n", yytext); else {
printf("Identifier: %s\n", yytext);
addToSymbolTable(yytext);
    }
}

[\\n]      { line++; }

[ \\t]+    ; // Ignore whitespace

{
printf("Unrecognized symbol: %s\n", yytext); }
%%
int isKeyword(char *str) {
    for (int i = 0; i < sizeof(keywords)/sizeof(keywords[0]); i++)
        { if (strcmp(str, keywords[i]) == 0)
            return 1;
          }
    return 0;
}

```

```

// --- Symbol Table Implementation ---
#define SIZE 100

```

```

char *symbolTable[SIZE];
int symCount = 0;

void addToSymbolTable(char *identifier) {
    for (int i = 0; i < symCount; i++) {
        if (strcmp(symbolTable[i], identifier) == 0)
            return; // Already exists
    }
    symbolTable[symCount++] = strdup(identifier);
}

```

```

// Optional: Print Symbol Table at End

```

```

int yywrap() {
    printf("\\n--- Symbol Table ---
\\n"); for (int i = 0; i < symCount;
i++)
        printf("%s\\n", symbolTable[i]);
    return 1;
}

```

Input (test.c)

```

int main() {
    float a = 3.14;
    int b = 10;
    // This is a comment a
    = a + b;
}

```

Execution Steps:

```

lex lexer.l
gcc lex.yy.c -o lexer
./lexer < test.c

```

Output:

Keyword: int

Identifier: main

Left Paren: (

Right Paren:) Left

Brace: { Keyword:

float Identifier: a

Operator: =

Float Constant: 3.14

Operator: ; Keyword:

int

Identifier: b

Operator: =

Integer Constant: 10

Operator: ;

Comment: // This is a comment

Identifier: a

Operator: =

Identifier: a

Operator: +

Identifier: b

Operator: ;

Right Brace: }

--- Symbol Table ---

main

a b

RESULT:

The lexical analyzer was successfully developed using LEX. It identified keywords, identifiers, constants, operators, and comments, and stored unique identifiers in a symbol table.

Ex 6.i, Implement a Lexical Analyzer using LEX Tool

AIM

To implement a Lexical Analyzer using the LEX tool that reads a source program and identifies tokens such as keywords, identifiers, numbers, and operators

ALGORITHM

1. Start the program.
2. Define regular expressions for tokens:
 - Keywords
 - Identifiers
 - Numbers
 - Operators
3. In the rules section, write actions for each regular expression.
4. Use yylex() to start lexical analysis.
5. For each matched token:
 - Print the token type and value.
6. Ignore whitespaces and newlines.
7. Stop when input ends (yywrap() returns 1).
8. **End** the program

PROGRAM:

```
%{
#include <stdio.h>
}%
// Definitions
KEYWORD    int|float|if|else|while|return
IDENTIFIER [a-zA-Z_][a-zA-Z0-9_]*
NUMBER     [0-9]+(\.[0-9]+)?
OPERATOR   \+|\-|\*|\/|\=|\<|\>|\<=\|\>=\|==
%%
// Rules
{KEYWORD}   { printf("Keyword: %s\n", yytext); }
{IDENTIFIER} { printf("Identifier: %s\n", yytext); }
{NUMBER}    { printf("Number: %s\n", yytext); }
{OPERATOR}  { printf("Operator: %s\n", yytext); }
[ \t\n]+    { /* Ignore whitespace */ }
.           { printf("Unknown token: %s\n", yytext); }
%%
// Main function
int main() {
    printf("Enter the code (Ctrl+D to end input):\n");
    yylex();
    return 0;
}
// Function to signal end of input
int yywrap() {
    return 1;
}
```

Steps to Compile and Run

1. Save the above code in a file named lexer.l.
2. Open terminal and run:

```
lex lexer.l          # Generates lex.yy.c
gcc lex.yy.c -ll -o lexer # Compiles the code
./lexer              # Run the program
```

Sample Input (entered after running ./lexer):

```
int num = 25;
float value = 3.14;
if (num > 10) return value;
Sample Output:
Keyword: int
Identifier: num
Operator: =
Number: 25
Operator: ;
Keyword: float
Identifier: value
Operator: =
Number: 3.14
Operator: ;
Keyword: if
Operator: (
Identifier: num
Operator: >
Number: 10
Operator: )
Keyword: return
Identifier: value
Operator: ;
```

Output:

```
Keyword: int
Identifier: main
Unknown token: (
Unknown token: )
Unknown token: {
Identifier: printf
Unknown token: (
Unknown token: "
Unknown token: E
Unknown token: n
Unknown token: t
Unknown token: e
Unknown token: r
Unknown token:
Unknown token: t
Unknown token: h
Unknown token: e
Unknown token:
Unknown token: c
Unknown token: o
Unknown token: d
Unknown token: e
Unknown token:
Unknown token: (
Unknown token: C
Unknown token: t
Unknown token: r
Unknown token: l
Unknown token: +
Unknown token: D
Unknown token:
Unknown token: t
Unknown token: o
Unknown token:
Unknown token: e
```

Unknown token: n
Unknown token: d
Unknown token:
Unknown token: i
Unknown token: n
Unknown token: p
Unknown token: u
Unknown token: t
Unknown token:)
Unknown token: :
Unknown token: \
Unknown token: n
Unknown token: "
Unknown token:)
Unknown token: ;
Identifier: yylex
Unknown token: (
Unknown token:)
Unknown token: ;
Keyword: return
Number: 0
Unknown token: }
Operator: /
Operator: /
Identifier: Function
Identifier: to
Identifier: signal
Identifier: end
Identifier: of
Identifier: input
Keyword: int
Identifier: yywrap
Unknown token: (
Unknown token:)
Unknown token: {
Keyword: return
Number: 1
Unknown token: }

Result:

The program successfully implements a Lexical Analyzer using the LEX Tool. It identifies and classifies different tokens in the input such as keywords, identifiers, numbers, and operators.

Ex.6,ii. Implementation of calculator using LEX and YACC

AIM

To design and implement a simple arithmetic expression calculator using LEX and YACC that supports basic operations: addition, subtraction, multiplication, division, and parentheses

ALGORITHM

1. Lexical Analysis (LEX) Identify tokens: numbers, operators (+, -, *, /), parentheses (,), and newline (\n). Return token types to the YACC parser.
2. Syntax Analysis (YACC) Define grammar rules for arithmetic expressions. Use operator precedence and Associativity. Evaluate the expression during parsing (semantic actions).
3. Execution Compile using lex and yacc. Input arithmetic expressions. Output the computed result. Exit on EOF (Ctrl+D or Ctrl+Z depending on OS).

PROGRAM

1. calc.l – LEX Program

```
%{
#include "y.tab.h"
#include <stdio.h>
}%

%%
[0-9]+ { yylval = atoi(yytext); return NUMBER; }
[+\\-*/] { return yytext[0]; }
[\\n] { return '\\n'; }
[(\\)] { return yytext[0]; }
[ \\t] { /* Ignore whitespace */ }
. { printf("Invalid character: %s\\n", yytext); }
%%

int yywrap() {
    return 1;
}
```

2. calc.y – YACC Program

```
%{
#include <stdio.h>
#include <stdlib.h>
}%

%token NUMBER
%left '+' '-'
%left '*' '/'
%right UMINUS

%%
input:
    /* empty */
    | input line
    ;

line:
    '\\n'
    | expr '\\n' { printf("Result = %d\\n", $1); }
    ;

expr:
```

```

NUMBER          { $$ = $1; }
| expr '+' expr  { $$ = $1 + $3; }
| expr '-' expr  { $$ = $1 - $3; }
| expr '*' expr  { $$ = $1 * $3; }
| expr '/' expr  {
    if ($3 == 0) {
        printf("Error: Division by zero\n");
        exit(1);
    }
    $$ = $1 / $3;
}
| '-' expr %prec UMINUS { $$ = -$2; }
| '(' expr ')'          { $$ = $2; }
;
%%

int main() {
    printf("Enter arithmetic expressions (Ctrl+D to exit):\n");
    yyparse();
    return 0;
}

int yyerror(const char *s) {
    fprintf(stderr, "Parse error: %s\n", s);
    return 1;
}

```

How to Compile and Run

```

yacc -d calc.y      # Generates y.tab.c and y.tab.h
lex calc.l          # Generates lex.yy.c
gcc y.tab.c lex.yy.c -o calc -ll # Compile and link LEX/YACC
./calc              # Run the calculator

```

OUTPUT:

Sample Input and Output

```

5 + 3
7 * (2 + 3)
10 / 2 - 1
-4 + 6
10 / 0

```

Output:

```

Result = 8
Result = 35
Result = 4
Result = 2
Error: Division by zero

```

Result:

The arithmetic calculator was successfully implemented using LEX and YACC. It correctly evaluates expressions with operator precedence, parentheses, and unary minus. Errors such as division by zero are also handled.

EX: 7 Implement Shift Reduce Parsing

AIM:

To implement Shift-Reduce Parsing for a given grammar and input string using C.

ALGORITHM:

1. Start with an empty stack and the input string.
2. **Shift**: Push the next input symbol onto the stack.
3. **Reduce**: If the top of the stack matches the RHS (Right-hand side) of any production, replace it with the LHS (Left-hand side).
4. Repeat Shift or Reduce operations until:
 - The stack contains only the start symbol and the input is fully read (**ACCEPT**), or
 - No valid shift or reduce is possible (**REJECT**).

Program

```
#include <stdio.h>
#include <string.h>
char stack[50]; char
input[50];
int top = -1, i = 0;
// Function to print stack and input
void printStackInput(int j) {
    printf("\nStack: ");
    for (int k = 0; k <= top; k++) printf("%c", stack[k]);
    printf("\nInput: ");
    for (int k = j; k < strlen(input); k++) printf("%c", input[k]);
}
// Function to reduce stack content int
reduce() {
    // Try reducing id (represented as 'i') to E if
    (top >= 0 && stack[top] == 'i') {
        stack[top] = 'E';
        printf("=> Reduced: E → id");
        return 1;
    }
}
```

```

// Try reducing  $E+E \rightarrow E$ 
if (top >= 2 && stack[top] == 'E' && stack[top-1] == '+' && stack[top-2] == 'E')
    { stack[top-2] = 'E';
      top -= 2;
      printf("=> Reduced:  $E \rightarrow$ 
         $E+E$ "); return 1;
    }
// Try reducing  $E * E \rightarrow E$ 
if (top >= 2 && stack[top] == 'E' && stack[top-1] == '*' && stack[top-2] == 'E')
    { stack[top-2] = 'E';
      top -= 2;
      printf("=> Reduced:  $E \rightarrow$ 
         $E * E$ "); return 1;
    }
// Try reducing  $(E) \rightarrow E$ 
if (top >= 2 && stack[top] == ')' && stack[top-1] == 'E' && stack[top-2] == '(')
    { stack[top-2] = 'E';
      top -= 2;
      printf("=> Reduced:  $E \rightarrow (E)$ ");
      return 1;
    }
return 0; // No reduction possible
}

int main() {
    printf("Enter the input string (like i+i*i): ");
    scanf("%s", input);
    printf("\nShift-Reduce Parsing Steps:\n");
    while (i <= strlen(input)) {
        printStackInput(i);

        if (i < strlen(input)) {
            // Shift

```

```

        stack[++top] = input[i++];
        printf(" => Shift");
    }

    // Keep reducing until not possible while
    (reduce());

}

// Final acceptance check
if (top == 0 && stack[0] == 'E') {
    printf("\n\nInput string ACCEPTED.\n");
} else {
    printf("\n\nInput string REJECTED.\n");
}

return 0;
}
OUTPUT:

```

Enter the input string (like i+i*i):

i+i*i Shift-Reduce Parsing Steps:

Stack: Input: i+i*i => Shift

Stack: i Input: +i*i => Reduced: E →

id Stack: E Input: +i*i =>

Shift

Stack: E+ Input: i*i => Shift

Stack: E+i Input: *i => Reduced: E → id

Stack: E+E Input: *i => Shift

Stack: E+E* Input: i => Shift

Stack: E+E*i Input: => Reduced: E → id

Stack: E+E*E Input: => Reduced: E →

E*E Stack: E+E Input: => Reduced: E

→ E+E Stack: E Input: =>

Input string ACCEPTED.

RESULT:

The input string is successfully parsed using Shift-Reduce Parsing and accepted according to the given grammar.

Ex:8 Implement Predictive Parsing algorithm

Aim

To implement a Predictive Parser (LL(1) parser) in C, construct the parsing table, and parse input strings to check whether they belong to the grammar or not.

Algorithm

1. **Initialize** stack with \$ and start symbol E.
2. Read input string (append \$).
3. Repeat until stack is empty:
 - o If top of stack is a terminal and matches input symbol $\rightarrow$ pop & advance.
 - o If top of stack is a terminal but does not match $\rightarrow$ error.
 - o If top is a non-terminal $\rightarrow$ consult parsing table and replace with production.
 - o If no entry $\rightarrow$ error.
4. Accept if both input and stack reach \$.

Program

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX 100

// Stack implementation
char stack[MAX];
int top = -1;

void push(char c) {
    if(top < MAX-1) stack[++top] = c;
}

char pop() {
    if(top >= 0) return stack[top--];
    return '\0';
}

char peek() {
    if(top >= 0) return stack[top];
    return '\0';
}

// Parsing Table for grammar:
// E  $\rightarrow$  T E'
// E'  $\rightarrow$  + T E' |  $\epsilon$ 
// T  $\rightarrow$  F T'
// T'  $\rightarrow$  * F T' |  $\epsilon$ 
// F  $\rightarrow$  (E) | i (we use 'i' for id)
const char* table[5][6];
// Non-terminals: E=0, Ep=1, T=2, Tp=3, F=4
// Terminals: i=0, +=1, *=2, (=3, )=4, $=5

int getNonTerminalIndex(char nt) {
    switch(nt) {
        case 'E': return 0;
        case 'Q': return 1; // we use Q for E'
        case 'T': return 2;
        case 'R': return 3; // we use R for T'
        case 'F': return 4;
    }
}
```

```

    }
    return -1;
}

int getTerminalIndex(char t) {
    switch(t) {
        case 'i': return 0;
        case '+': return 1;
        case '*': return 2;
        case '(': return 3;
        case ')': return 4;
        case '$': return 5;
    }
    return -1;
}

int main() {
    // Fill parsing table
    // Using Q instead of E' and R instead of T' (to avoid confusion with prime symbol)

    // E
    table[0][0] = "TQ"; // E -> TQ for i
    table[0][3] = "TQ"; // E -> TQ for (

    // Q (E')
    table[1][1] = "+TQ"; // Q -> +TQ
    table[1][4] = "e"; // Q ->  $\epsilon$ 
    table[1][5] = "e"; // Q ->  $\epsilon$ 

    // T
    table[2][0] = "FR"; // T -> FR
    table[2][3] = "FR"; // T -> FR

    // R (T')
    table[3][1] = "e"; // R ->  $\epsilon$ 
    table[3][2] = "*FR"; // R -> *FR
    table[3][4] = "e"; // R ->  $\epsilon$ 
    table[3][5] = "e"; // R ->  $\epsilon$ 

    // F
    table[4][0] = "i"; // F -> i
    table[4][3] = "(E)"; // F -> (E)

    char input[MAX];
    printf("Enter input string (use i for id): ");
    scanf("%s", input);
    strcat(input, "$");

    // Initialize stack
    push('$');
    push('E');

    int ip = 0;
    printf("\n%-20s %-20s %-20s\n", "Stack", "Input", "Action");
    while(top >= 0) {
        char X = peek();
        char a = input[ip];

        // Print current config
        for(int i=0; i<=top; i++) printf("%c", stack[i]);
        printf("%-20s ", "");
    }

```

```

printf("%-20s ", input+ip);

if(X == a && X == '$') {
    printf("Accept\n");
    break;
}
else if(X == a) {
    pop();
    ip++;
    printf("Match %c\n", a);
}
else if(X=='E' || X=='Q' || X=='T' || X=='R' || X=='F') {
    int row = getNonTerminalIndex(X);
    int col = getTerminalIndex(a);
    const char* prod = table[row][col];
    if(prod == NULL) {
        printf("Error: no rule\n");
        break;
    }
    pop();
    if(strcmp(prod,"e")!=0) {
        for(int i=strlen(prod)-1;i>=0;i--) push(prod[i]);
    }
    printf("%c -> %s\n", X, prod);
}
else {
    printf("Error\n");
    break;
}
}
return 0;
}

```

output

Stack	Input	Action
\$E	i+i*i\$	E -> TQ
\$Q T	i+i*i\$	T -> FR
\$Q R F	i+i*i\$	F -> i
\$Q R i	i+i*i\$	Match i
\$Q R	+i*i\$	R -> e
\$Q	+i*i\$	Q -> +TQ
\$Q T +	+i*i\$	Match +
\$Q T	i*i\$	T -> FR
\$Q R F	i*i\$	F -> i
\$Q R i	i*i\$	Match i
\$Q R	*i\$	R -> *FR
\$Q R F *	*i\$	Match *
\$Q R F	i\$	F -> i
\$Q R i	i\$	Match i
\$Q R	\$	R -> e
\$Q	\$	Q -> e
\$	\$	Accept

Result

The input string i+i*i is successfully parsed using the Predictive Parsing Algorithm in C, proving that the algorithm works for LL(1) grammar parsing.

EX:9 Implement Intermediate code generation Postfix, Prefix (Infix to Postfix)

AIM

To implement a program for intermediate code generation using Infix to Postfix and Infix to Prefix conversion.

ALGORITHM

Infix to Postfix

1. Initialize an empty stack.
2. Scan the infix expression from left to right.
3. If operand, add it to output.
4. If '(', push to stack.
5. If ')', pop till '(' is found.
6. If operator, pop from stack to output until precedence condition fails, then push the operator.
7. After input ends, pop remaining stack content to output.

Infix to Prefix

1. Reverse the infix expression.
2. Swap '(' with ')'.
3. Apply infix to postfix logic.
4. Reverse the result to get

prefix.

PROGRAM:

```
#include<stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define SIZE 100 char
stack[SIZE]; int top = -
1;
// Push operation void
push(char c) {
stack[++top] = c;
}
// Pop operation char
pop() {
return stack[top--];
```

```

}

// Stack top char
peek() {
return stack[top];
}

// Check if operator int
isOperator(char c) {
return (c == '+' || c == '-' || c == '*' || c == '/');
}

// Precedence of operators int
precedence(char op) {
if (op == '+' || op == '-') return 1; if (op ==
'*' || op == '/') return 2; return 0;
}

// Infix to Postfix
void infixToPostfix(char infix[], char postfix[]) { int i, j
= 0;
char c;
for (i = 0; i < strlen(infix); i++) { c
= infix[i];
if (isspace(c)) continue; if
(isalnum(c)) {
postfix[j++] = c;
}
else if (c == '(') {
push(c);
}
else if (c == ')') {
while (top != -1 && peek() != '(')
postfix[j++] = pop();
pop(); // remove '('
}
else if (isOperator(c)) {
while (top != -1 && precedence(peek()) >= precedence(c))
postfix[j++] = pop();
push(c);
}
}
}

```

```

    }

    }
    while (top != -1) postfix[j++] =

        pop();
postfix[j] = '\0';
}

// Reverse string
void reverse(char str[]) { int n =
    strlen(str);

    for (int i = 0; i < n / 2; i++) { char
        tmp = str[i];
        str[i] = str[n - i - 1];
        str[n - i - 1] = tmp;
    }
}

// Swap ( and )
void swapParentheses(char expr[]) { for (int i =
    0; i < strlen(expr); i++) {
        if (expr[i] == '(') expr[i] = ')';
        else if (expr[i] == ')') expr[i] = '(';
    }

} // Infix to Prefix
void infixToPrefix(char infix[], char prefix[]) { char
    revInfix[SIZE], postfix[SIZE];
    strcpy(revInfix, infix);
    reverse(revInfix);
    swapParentheses(revInfix);

    top = -1;
    infixToPostfix(revInfix, postfix); reverse(postfix);
    strcpy(prefix, postfix);
}
// MAIN FUNCTION

int main() {

    char infix[SIZE], postfix[SIZE], prefix[SIZE];
    printf("Enter Infix Expression: "); fgets(infix,
    SIZE, stdin);

    infix[strlen(infix, "\n")] = '\0'; // remove newline infixToPostfix(infix,

```

```
postfix);  
infixToPrefix(infix, prefix);  
printf("\nPostfix Expression: %s\n", postfix);  
printf("Prefix Expression : %s\n", prefix); return 0;  
}
```

OUTPUT

Enter Infix Expression: (A+B)*C

Postfix Expression: AB+C* Prefix

Expression : *+ABC

RESULT:

Intermediate code generation using Infix to Postfix and Infix to Prefix conversion was successfully implemented

